Godot 4.7.2 · GDScript · 26 scripts · 8 scenes · 137 traced requirements · 44 ADRs

CodeQuest

A Game-Based Learning System for Software Engineering Education

B Haris Ahamed · 24BCE1136
Shaili H · 24BCE1970

BCSE301L — Software Engineering · Digital Assignment 2

OVERVIEW

Problem, design response, engineering objective

PROBLEM

Process knowledge has no feedback loop

Code compiles or it does not; a test passes or fails. Feedback arrives in seconds. A requirement gets a mark weeks later, so nothing in between makes a badly written requirement cost anything. The observable result is terminology reproduced accurately and applied poorly.

DESIGN RESPONSE

The assessment instrument is the mechanic

There is no attack button. A correct answer is the hero's attack; an incorrect answer is the General's. Because the quiz is not separable from the game, this is content gamification rather than points and badges wrapped around unchanged material.

Primary engineering objective — adding a world must not require changing gameplay code

Satisfied by a three-layer architecture with one-way dependencies (Section 7). A world is one .tres resource, a block of rows in questions.csv and one JSON lesson file; no gameplay script is edited to add the ninth.

8	26	129	1159
worlds, one per SDLC phase	GDScript modules	authored questions	test assertions, 0 failures

PART ONE

Requirements and design

The requirements baseline, the use-case model, the layered architecture and the data flow

REQUIREMENTS & DESIGN

A 137-item requirements baseline, traced to components

114	6	6	9
Met — implemented and verified	Partial — remainder stated in the row	Diverged — a named ADR gives the reason	Open — eight of them diagrams, now delivered

GROUP	REPRESENTATIVE REQUIREMENT	VERIFIED BY
FR-4 Battle	Each question shall be drawn without repetition from that world's pool; an explanation shall follow every answer, correct or not.	Automated check 3 over the question bank; manual cases M11–M14
FR-6 Progression	Clearing a world shall reduce the Bug Lord's health by exactly 1000; clearing an already-cleared world shall have no effect.	Automated check 4 — progression arithmetic, headless
FR-8 Persistence	The save shall carry a version field and shall refuse a file of a different version rather than apply it partially.	Automated check 5; manual case M22
NFR-5 Reliability	The battle state machine shall not accept an answer outside the QUESTION state, so a repeated input cannot resolve twice.	State gate asserted in code; manual case M13
NFR-6 Maintainability	No script shall carry unrelated responsibilities; content shall be addable without modifying gameplay code.	Automated check 1 — layer dependency scan

Use-case model — two actors, one system boundary

Two actor classes, deliberately asymmetric

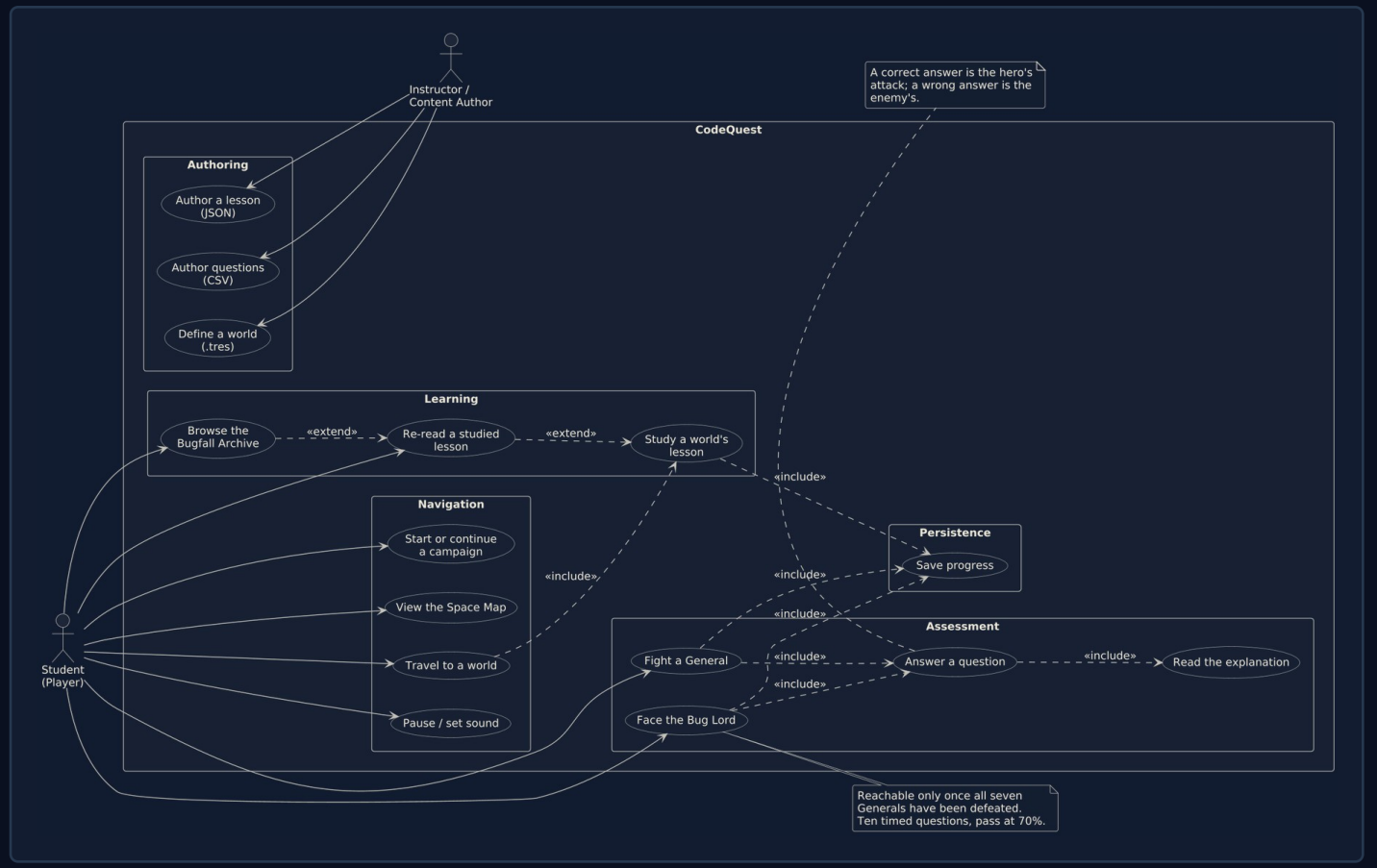
The student plays; the content author edits .tres, .csv and .json files and never launches the game. Authoring therefore sits outside the system boundary, which is why no authoring UI is specified.

Fifteen use cases in five packages

Navigation, Learning, Assessment, Persistence and Authoring. The packages are the same partition the architecture uses, so a use case maps to one layer rather than cutting across several.

Save progress is an «include», not a step

Persistence is pulled into clearing a world rather than offered as a player action. There is no save menu, and no path by which progress can be made and then lost.



Use-case diagram — student and content author against the CodeQuest boundary

REQUIREMENTS & DESIGN

Three layers, strictly one-way dependencies

Presentation

may depend on Session and Data

Eight scenes and their controllers — Main, SpaceMap, RocketTravel, Lesson, Battle, QuestionPanel, Library, Victory

Session

may depend on Data only

Four autoloads — GameState, SceneRouter, AudioManager, PauseMenu — plus the WorldFlow transition function

Data

depends on nothing

WorldData · WorldDatabase · EnemyData · HeroData · QuestionData · QuestionBank · LessonBank

HOW IT IS ENFORCED

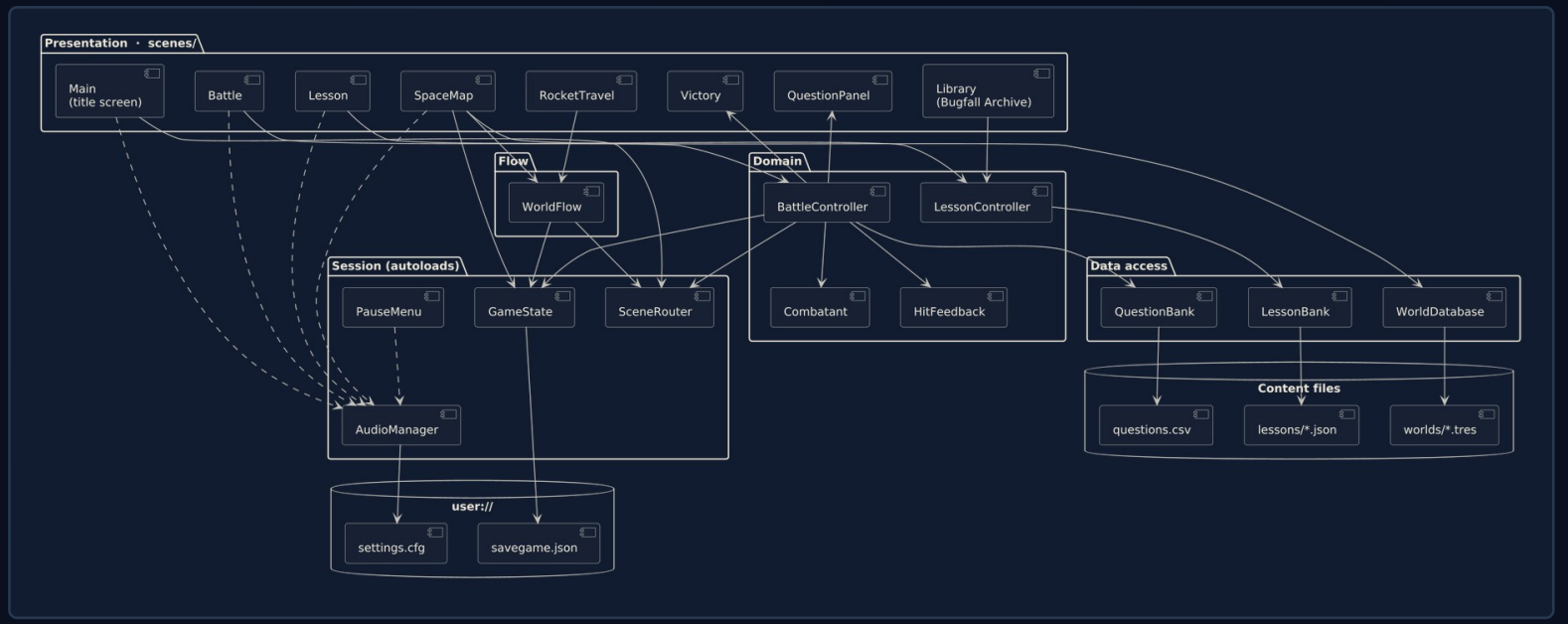
Automated check 1 scans every script for an import that crosses a layer upwards and fails the build if it finds one. The rule is not a convention in a document; it is a test.

Consequence: a new world is three content files. Nothing in the Session or Presentation layer is opened to add one, because neither layer names a world.

44 architecture decision records

Every significant choice — the layer rule, the four autoloads, the single flow function, the three content formats — is recorded with its alternatives and the reason it was rejected, in docs/DECISIONS.md.

Component model — nineteen components, five packages

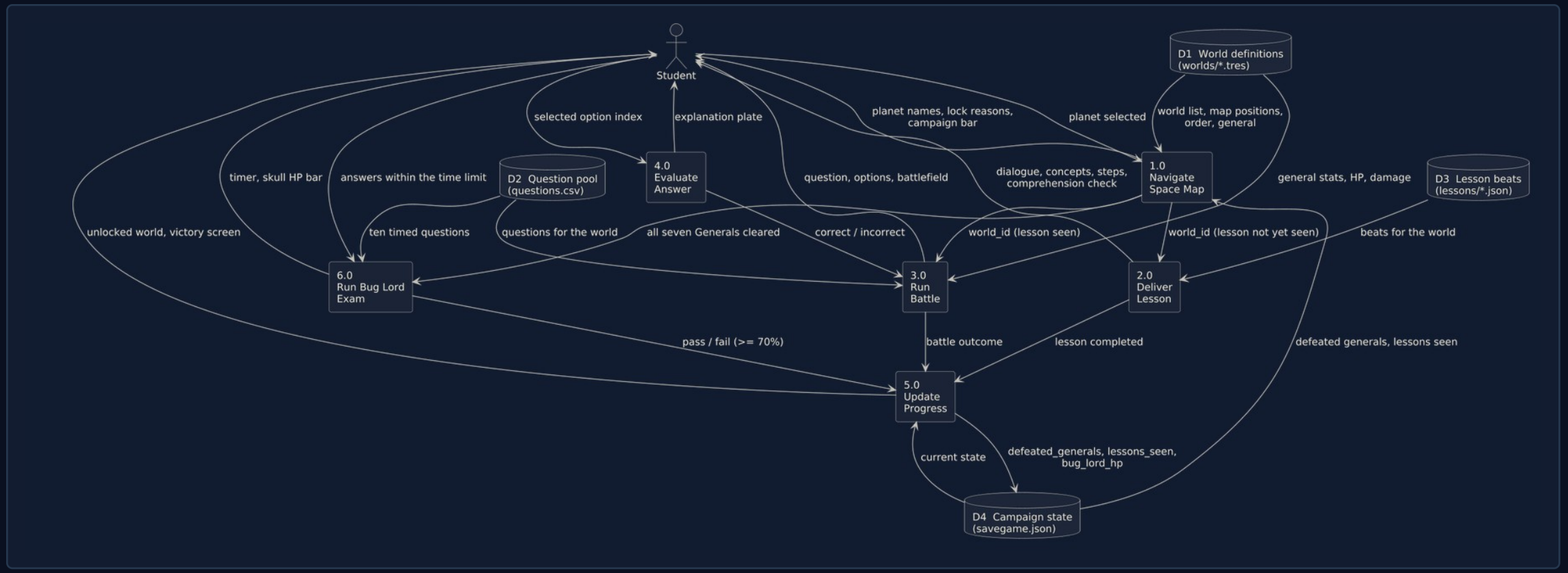


Components grouped by layer; the two user:// stores are the only writable ones

Read the arrows, not the boxes

Every arrow points downwards or sideways within a layer; none points up. AudioManager is reached from every scene, which is the justification for it being an autoload rather than a node — a non-autoload would have to be found or re-instantiated by each of the eight scenes. settings.cfg and savegame.json are the only two writable stores in the system, and only GameState writes the second.

Data flow — one process writes campaign state



Level 1 data flow — six processes, four data stores

Process 5.0 is the only process that writes campaign state — so a save cannot be written from two places and cannot be written inconsistently. **The three content stores are read-only at runtime** — the content a student plays cannot be altered by playing it.

PART TWO

Implementation

The technology stack, the content pipeline, the battle state machine and the principal algorithms

IMPLEMENTATION

Technology stack, and the reason for each choice

LAYER	TECHNOLOGY	REASON IT WAS CHOSEN OVER THE ALTERNATIVE
Engine	Godot 4.7.2 stable	Open source with no licence cost or revenue share; a scene-and-node model that maps directly onto the layered design; first-class 2D rather than 2D bolted onto a 3D pipeline
Language	GDScript, statically typed	Native to the engine's signal and resource systems; static typing throughout so type errors surface at compile time rather than in a battle
Renderer	OpenGL 3.3 Compatibility	Runs on integrated graphics. Forward+ offers nothing a 2D game uses and raises the hardware floor for a classroom
Persistence	JSON in user://, schema v1	Human-readable for debugging; a version field lets an incompatible save be refused outright instead of applied in part
Asset generation	Python 3 — NumPy, SciPy, Pillow	The 16 sound effects and 5 music loops are synthesised by script, so they are original work and regenerable rather than sourced
Verification	Godot headless, custom runner	Five checks, 1159 assertions, no external framework — a dependency whose only user is five files is not worth its own failure mode

IMPLEMENTATION

Three content formats, selected by editing profile

STRUCTURE 1

Godot Resource (.tres)

Worlds, Generals and the hero. Edited in the inspector, referenced by other resources, diffable as text. 41 SpriteFrames resources carry the animation data.

TABULAR BULK 2

CSV

129 questions of identical flat shape: text, four options, correct index, explanation, time limit. A spreadsheet is the right tool for 129 identical rows, and an instructor already has one.

NESTED AUTHORING 3

JSON

Lessons. A beat is polymorphic — a compare beat and a steps beat carry entirely different fields — which CSV cannot express without a column per field of every type.

The defect this pipeline produced, and how it was found

Godot's import system claims .csv for translations and .json for its own resource pipeline, so neither raw file is packed into an exported build. The editor is perfectly happy: the files are on disk and load correctly. An exported game would have shipped with no questions and no lessons. Both formats now carry a sibling .import file specifying importer="keep". The defect was found by reading the importer's documentation, not by running the game — no test that passes in the editor could have caught it.

One state machine drives both encounter types



BattleController states

Five states, one legal entry point for input

INTRO → QUESTION → RESOLVING → ENDING → FINISHED. An answer is accepted only in QUESTION; the controller returns immediately in every other state. A double click, or a number key pressed while the hero's attack is still playing, cannot resolve the same question twice (NFR-5).

Two encounters, one loop, one parameter object

A General fight is attrition: health against health, ending when either reaches zero. The Bug Lord is a timed pass-or-fail examination of ten questions. Both run this machine; a BattleRequest selects the timer, the question count, the layout and the ending condition. The second encounter type cost a parameter object, not a second controller.

The one sequence that is explicit

The Ultimate — seven beats of flash, collapse, twenty-four slash arcs over 2.5 s and a violet burst — is scripted rather than emergent, because it is a single scripted reward and not a repeating state.

IMPLEMENTATION

Principal algorithms and the invariant each preserves

ALGORITHM	METHOD	INVARIANT PRESERVED	ALTERNATIVE REJECTED
World unlocking	Unlocked when order index ≤ cleared.size(). Evaluated on demand, O(1), nothing stored.	The unlock set is always exactly consistent with the cleared set, because it is derived from it.	A per-world unlocked flag — a second copy of the same fact, free to drift from the first.
Question selection	The pool is drawn into a used list; each draw samples uniformly from the complement.	No question recurs within an encounter, and the order still differs between playthroughs.	Sampling with replacement, which repeats inside one fight; pre-shuffling, which makes replays identical.
Progression update	clear_world(id) appends to cleared only if the identifier is absent, then saves.	Clearing is idempotent: replaying a cleared world cannot advance the campaign twice.	An unguarded append or a counter increment, either of which inflates progress on a replay.
Bug Lord health	Derived at battle start as 8000 − 1000 × cleared.size(). Never stored.	Boss health cannot disagree with campaign progress, and inherits idempotency from the clause above.	A mutable boss-health field, which must then be kept in step with every progression write.
Animation timing	Duration computed as frame count ÷ frame rate rather than awaited from a signal.	Timing is correct for all 41 SpriteFrames resources, every one of which is set to loop.	await animation_finished — Godot never emits it for a looping animation, so the sequence deadlocks.

IMPLEMENTATION

Instruction is delivered before any assessment

Seven beat types, one renderer each

Dialogue, concept, compare, example, steps, check and encounter. Beats stay dictionaries rather than becoming seven typed classes: seven classes with disjoint fields would add a hierarchy and remove nothing.

Navigable in both directions

A student who has forgotten Reqoria's terminology in world five can step back through the lesson or re-read it from the Bugfall Archive, which marks which of the eight have been studied.

A check inside the lesson

A comprehension question with the reasoning revealed, before the General is ever seen. The ordering — LEARN, UNDERSTAND, APPLY, TEST, PROGRESS — is the pedagogical claim the design makes.



A compare beat: functional against non-functional requirements

IMPLEMENTATION

The battle: the question is the turn

Answer, not attack

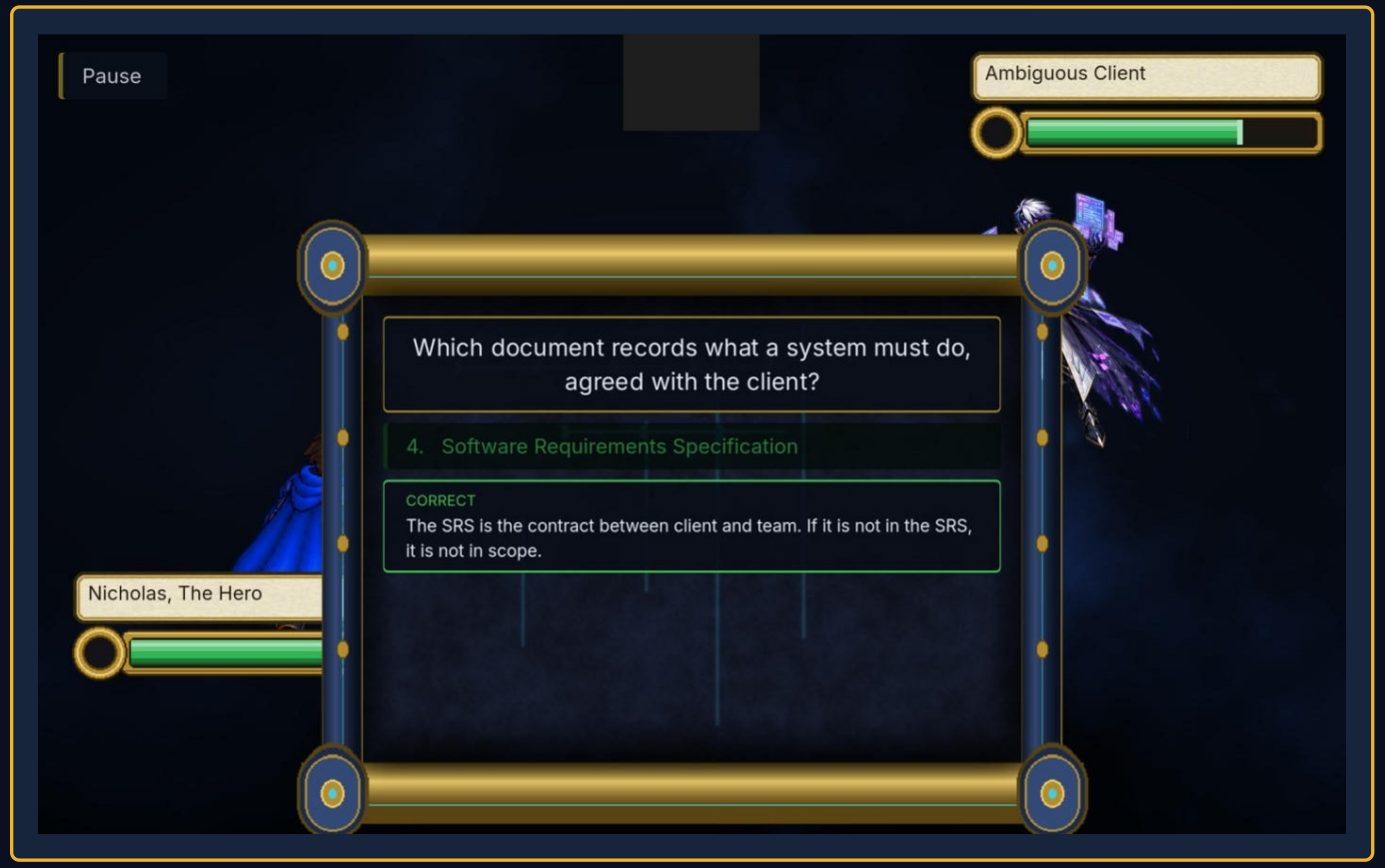
Correct, the hero strikes for the hero's attack damage; incorrect, the General strikes for its own. Mouse or number keys 1–4. The encounter ends when either combatant reaches zero (FR-4.6).

Every answer teaches

The explanation names the correct option and the reason, whether the player was right or wrong. A failed question still instructs, which is what separates this from a graded quiz.

A measured defect, not a judged one

Instrumenting the panel showed the explanation overflowing its scroll area by 53 pixels in every battle in the game. Nobody had reported it: the text was simply clipped. Hiding the unchosen options was the fix.



Answering reveals the explanation; unchosen options are hidden to make room

IMPLEMENTATION

The Bug Lord — a gated examination, not a boss fight

Gated by progression, not by difficulty

The encounter is reachable only once all seven Generals are defeated (FR-6.3), and the boss's health is derived from the cleared set rather than stored. The campaign bar on the Space Map is that derivation, rendered.

One failure path, not two

An expired timer is recorded as an incorrect answer, so the state machine never has to distinguish timeout from error. The hero's health is sized so the fourth wrong answer empties it exactly — failing the examination and losing the fight coincide by construction.

The pause defect that this exposed

Godot's create_timer defaults to process_always = true, so a paused game kept counting down. All eleven calls in the project were corrected; the examination clock now stops with everything else.



8000
starting health

−1000
per world cleared

70%
of ten to pass

PART THREE

Verification

The test strategy, what the defect log showed, and what the traceability matrix records

VERIFICATION

Five automated checks, selected by risk exposure

Selection criterion — probability of the defect class × cost of missing it

Coverage was not the target. A 2D game's animation and feel cannot be meaningfully asserted at this size, and a check that tests them would cost more than it returns; data integrity and progression arithmetic can be asserted exactly, headlessly, in under a second, and a defect in either is fatal to the whole campaign.

RISK	PROBABILITY ASSESSED	COST IF MISSED	CHECK
Content errors	Medium — 129 hand-authored rows	A game that teaches something false	3 — bank integrity: index ranges, option counts, duplicates
Editor-only correctness	High — two already found	An exported game with no questions and no lessons	2 — import flags and resource packing
Progression corruption	Low — clear_world is guarded	The campaign unfinishable from world one	4 — progression arithmetic and idempotency
Layer violation	Medium — 26 scripts, two authors	The content-addable property silently lost	1 — dependency direction scan
Save incompatibility	Low — schema versioned	A partially applied save, worse than none	5 — version refusal and round-trip

VERIFICATION

What the defect log showed about where defects live

Five of fourteen were the engine behaving correctly

1

...in a way the design did not anticipate: looping animations that never emit animation_finished, timers that run while the tree is paused, a VBoxContainer that re-lays out its children and fights a position tween. No unit test would have predicted any of them, because none of them is a fault. They were found by running the system and watching it.

Two would appear only in an exported build

2

The .csv and .json import claims. Both were found by reading the engine's importer documentation while writing the deployment diagram. The editor reported nothing wrong because nothing was wrong in the editor — which is the entire category of defect that an editor-only test strategy cannot see.

One test reported a pass having asserted nothing

3

Run with --script, autoloads do not exist, so the script failed to compile and the runner reported success. A test that silently skips itself is worse than no test: it converts a known unknown into a false assurance. The runner now fails on a zero-assertion run.

The log is the evidence for the strategy, not a postscript to it

Three of these five categories — engine behaviour, export-only defects and silent test failure — are invisible to assertion count. That is the measured reason the strategy weights a short manual suite and a read of the engine's own documentation alongside the five automated checks, rather than pursuing coverage.

VERIFICATION

Technical contributions, and what remains

CONTRIBUTION 1

A transition function, not scattered routing

WorldFlow reduces every transition in the campaign to one pure function of game state. Eight scenes ask it what comes next; none of them knows. Adding the ninth world changes no scene.

CONTRIBUTION 2

A content pipeline chosen by editing profile

Three formats, each selected against a named alternative, with the engine's import behaviour handled explicitly. An instructor edits a spreadsheet; a developer edits a resource.

CONTRIBUTION 3

Idempotent progression by derivation

Unlock state and boss health are derived from the cleared set rather than stored beside it, so the two cannot disagree. One guarded append is the whole persistence-correctness argument.

CONTRIBUTION 4

Two encounter types, one state machine

An attrition fight and a timed pass-or-fail examination run the same five states, parameterised by a request object. The second encounter type cost a parameter object, not a controller.

None of the four depends on the subject being software engineering

Replace the eight .tres worlds, the question bank and the lesson files and the gameplay code is untouched. Next: verify against an exported build (manual cases M25–M28, since two logged defects were export-only), aggregate per-question failure rates across a class from the save file, and evaluate with a pre-test, post-test and delayed post-test design.

Thank you

Questions welcome

B Haris Ahamed · 24BCE1136 Shaili H · 24BCE1970

Project report · 44 ADRs · 137-item traceability matrix · 12 UML and SE diagrams